

Script to validate points in ReSurvey database using RS data (S2 + SatEmb + Landsat)

Fitting Random Forest models with ntree = 1500

Alicia Valdés

18 marzo 2026

This R script is used to validate the points in the ReSurvey database using either RS indicators from S2 or Landsat (indices + phenology + canopy height), or bands from the Google Satellite Embedding dataset: https://developers.google.com/earth-engine/datasets/catalog/GOOGLE_SATELLITE_EMBEDDING_V1_ANNUAL#description

Load libraries

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.0      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.2      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at C:/Users/jimenezalfaro/OneDrive - Universidad de Oviedo/IMIB/Analyses/MOTIVATE/MOTIVATION
```

```
library(parallel)
library(doParallel)
```

```
## Cargando paquete requerido: foreach
##
## Adjuntando el paquete: 'foreach'
##
## The following objects are masked from 'package:purrr':
##
##   accumulate, when
##
## Cargando paquete requerido: iterators
```

```
library(moreparty)
```

```
## Cargando paquete requerido: party
## Cargando paquete requerido: grid
## Cargando paquete requerido: mvtnorm
## Cargando paquete requerido: modeltools
## Cargando paquete requerido: stats4
## Cargando paquete requerido: strucchange
## Cargando paquete requerido: zoo
##
## Adjuntando el paquete: 'zoo'
##
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric
##
## Cargando paquete requerido: sandwich
##
## Adjuntando el paquete: 'strucchange'
##
## The following object is masked from 'package:stringr':
##
##   boundary
##
## Adjuntando el paquete: 'party'
##
## The following object is masked from 'package:dplyr':
##
##   where
```

```
library(caret)
```

```
## Cargando paquete requerido: lattice
##
## Adjuntando el paquete: 'caret'
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Adjuntando el paquete: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Adjuntando el paquete: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(rlang)
```

```
##
## Adjuntando el paquete: 'rlang'
##
## The following objects are masked from 'package:purrr':
##
##     flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##     flatten_raw, invoke, splice
```

```
library(stringr)
library(beepr)
library(permimp)
library(purrr)
library(gghalves)
library(patchwork)
library(scales)
```

```
##
## Adjuntando el paquete: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

Load functions

```
knitr::purl("functions_20260225.Rmd", output = "functions_20260225.R")
```

```
## [1] "functions_20260225.R"
```

```
source("functions_20260225.R")
```

Load previously created objects

```
# Define the folder path
folder_path <- here("objects", "RF_1500_20260318")

# List all .RData or .rda files in the folder
rdata_files <- list.files(folder_path, full.names = TRUE)

# Load each file
lapply(rdata_files, load, envir = .GlobalEnv)
```

```
## [[1]]
## [1] "filtered_data_2_Landsat_with_probs"
##
## [[2]]
## [1] "filtered_data_2_Landsat_with_probs_2x"
##
## [[3]]
## [1] "filtered_data_months_Landsat_with_probs"
##
## [[4]]
## [1] "filtered_data_months_Landsat_with_probs_2x"
##
## [[5]]
## [1] "filtered_data_slope_Landsat_with_probs"
##
## [[6]]
## [1] "filtered_data_slope_Landsat_with_probs_2x"
##
## [[7]]
## [1] "rf_2_Landsat"
##
## [[8]]
## [1] "rf_2_Landsat_prediction"
##
## [[9]]
## [1] "rf_2_Landsat_refin_20_10_2x"
##
## [[10]]
## [1] "rf_2_Landsat_refin_20_10_2x_prediction"
##
## [[11]]
## [1] "rf_2_Landsat_refin_20_10"
##
## [[12]]
## [1] "rf_2_Landsat_refin_20_10_prediction"
##
## [[13]]
```

```
## [1] "rf_months_Landsat"
##
## [[14]]
## [1] "rf_months_Landsat_prediction"
##
## [[15]]
## [1] "rf_months_Landsat_refin_20_10_2x"
##
## [[16]]
## [1] "rf_months_Landsat_refin_20_10_2x_prediction"
##
## [[17]]
## [1] "rf_months_Landsat_refin_20_10"
##
## [[18]]
## [1] "rf_months_Landsat_refin_20_10_prediction"
##
## [[19]]
## [1] "rf_slope_Landsat"
##
## [[20]]
## [1] "rf_slope_Landsat_prediction"
##
## [[21]]
## [1] "rf_slope_Landsat_refin_20_10_2x"
##
## [[22]]
## [1] "rf_slope_Landsat_refin_20_10_2x_prediction"
##
## [[23]]
## [1] "rf_slope_Landsat_refin_20_10"
##
## [[24]]
## [1] "rf_slope_Landsat_refin_20_10_prediction"
```

Read data

```
data_validation_Landsat <- read_tsv(here(
  "data", "clean", "final_RS_data_bands_Landsat_all_20260309.csv"))
```

```
## Rows: 121494 Columns: 119
## -- Column specification -----
## Delimiter: "\t"
## chr (4): EUNIS1, EUNIS2, biogeo, unit
## dbl (115): PlotObservationID, EVI_auc, EVI_auc_mar_oct, EVI_auc_pos2, EVI_av...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

No parsing issues!

```
nrow(data_validation_Landsat)
```

```
## [1] 121494
```

Load Resurvey db

```
ReSurvey_data <- read_csv(  
  here("data", "clean", "ReSurvey_QRST_data_V2025.2_20260304.csv")) %>%  
  select(PlotObservationID, EUNIS1, EUNIS2, `Location method`) %>%  
  rename(Lctnmth = `Location method`)
```

```
## Rows: 133804 Columns: 56  
## -- Column specification -----  
## Delimiter: ","  
## chr   (33): Country, ReSurvey project, ReSurvey site, ReSurvey plot, ReSurvey...  
## dbl   (13): PlotObservationID, Lon_updated, Lat_updated, year, n_ESY_EUNIS, P...  
## lgl   (9): ESY_EUNISa_4, ESY_EUNISc_1, ESY_EUNISc_2, ESY_EUNISc_3, ESY_EUNIS...  
## date  (1): date  
##  
## i Use 'spec()' to retrieve the full column specification for this data.  
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Define lists of predictor vars

Moments with slope method

```
# Excluding EVI because many values outside of range -1, 1  
# Not using canopy height!  
vars_RF_slope_Landsat <- c(  
  # Min values of all indices  
  "NDVI_min", "NDMI_min", "NDWI_min", "SAVI_min",  
  # Max values of NDMI and NDWI  
  "NDMI_max", "NDWI_max",  
  # AUC of NDVI and SAVI  
  "NDVI_auc", "SAVI_auc",  
  # Values of NDVI and SAVI at sos, pos and eos  
  "NDVI_sos_value", "NDVI_pos_value", "NDVI_eos_value",  
  "SAVI_sos_value", "SAVI_pos_value", "SAVI_eos_value",  
  # Differences pos-sos in value and doy  
  "NDVI_diff_pos_sos_value", "SAVI_diff_pos_sos_value",  
  "NDVI_diff_pos_sos_doy", "SAVI_diff_pos_sos_doy",  
  # Differences pos-eos in value and doy  
  "NDVI_diff_pos_eos_value", "SAVI_diff_pos_eos_value",  
  "NDVI_diff_pos_eos_doy", "SAVI_diff_pos_eos_doy",  
  # Growing season duration  
  "NDVI_gsd", "SAVI_gsd")
```

Months method

```
# Excluding EVI because many values outside of range -1, 1
# Not using canopy height!
vars_RF_months_Landsat <- c(
  # Min values of all indices
  "NDVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI, and SAVI between march and oct
  "NDVI_auc_mar_oct", "SAVI_auc_mar_oct",
  # Values of NDVI and SAVI at march, pos and oct
  "NDVI_avg_value_03", "NDVI_pos_value", "NDVI_avg_value_10",
  "SAVI_avg_value_03", "SAVI_pos_value", "SAVI_avg_value_10",
  # Differences pos-march in value and doy
  "NDVI_diff_pos_march_value", "SAVI_diff_pos_march_value",
  "NDVI_diff_pos_march_doy", "SAVI_diff_pos_march_doy",
  # Differences pos-oct in value and doy
  "NDVI_diff_pos_oct_value", "SAVI_diff_pos_oct_value",
  "NDVI_diff_pos_oct_doy", "SAVI_diff_pos_oct_doy")
```

POS-2 and POS+2 method

```
# Excluding EVI because many values outside of range -1, 1
# Not using canopy height!
vars_RF_2_Landsat <- c(
  # Min values of all indices
  "NDVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI and SAVI between pos_minus60 and pos_plus_60
  "NDVI_auc_pos2", "SAVI_auc_pos2",
  # Values of NDVI and SAVI at pos_minus60, pos and pos_plus_60
  "NDVI_pos_minus60_value", "NDVI_pos_value", "NDVI_pos_plus60_value",
  "SAVI_pos_minus60_value", "SAVI_pos_value", "SAVI_pos_plus60_value",
  # Differences pos-pos_minus60 in value
  "NDVI_diff_pos_minus60_value", "SAVI_diff_pos_minus60_value",
  # Differences pos-pos_plus_60 in value
  "NDVI_diff_pos_plus60_value", "SAVI_diff_pos_plus60_value")
```

Filter data to use in RF models

Function

```
filter_data_Landsat <- function(data, variables) {
  data %>%
    mutate(EUNIS1 = as.factor(EUNIS1)) %>%
    # Remove rows with missing values in predictors
```

```

dplyr::filter(if_all(all_of(variables), ~ !is.na(.))) %>%
# Remove all rows with wrong values of indices (not between -1 and 1)
dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
# With Landsat data, many observations removed because of high/low EVI values
# In Landsat_extraction, values of EVI outside of expected range were set as NA,
# so better not to use EVI here
# dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
# Select only variables needed
dplyr::select(PlotObservationID, EUNIS1, all_of(variables), Lctnmth,
              valid_1) %>%
# Keep only rows with differences > 0 in variables
filter(if_all(any_of(variables[grepl("diff", variables)]),
              ~ .x > 0)) %>%
# Select only GPS points
dplyr::filter(Lctnmth == "Location with GPS" |
              Lctnmth == "Location with differential GPS")
}

```

Filtered data

Select only GPS points that have passed the rough validation in each case.

```

filtered_data_slope_Landsat <- data_validation_Landsat %>%
# Rough validation based on NDWI
rough_validation_Landsat() %>%
# Join ReSurvey data to get Location method
left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
# Data filtering
filter_data_Landsat(vars_RF_slope_Landsat) %>%
# From GPS points, filter based on NDVI percentiles
filter_by_group_percentiles("NDVI_pos_value")

```

Joining with 'by = join_by(PlotObservationID)'

```

filtered_data_months_Landsat <- data_validation_Landsat %>%
# Rough validation based on NDWI
rough_validation_Landsat() %>%
# Join ReSurvey data to get Location method
left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
# Data filtering
filter_data_Landsat(vars_RF_months_Landsat) %>%
# From GPS points, filter based on NDVI percentiles
filter_by_group_percentiles("NDVI_pos_value")

```

Joining with 'by = join_by(PlotObservationID)'

```

filtered_data_2_Landsat <- data_validation_Landsat %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Data filtering
  filter_data_Landsat(vars_RF_2_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

## Joining with 'by = join_by(PlotObservationID)'

```

(Appendix 2) Plots distribution max NDVI

```

plot_perc_Landsat <- data_validation_Landsat %>%
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  # Data filtering
  filter_data_Landsat(vars_RF_months_Landsat) %>%
  distr_plot("NDVI_pos_value", "Maximum NDVI", grey = T)

## Joining with 'by = join_by(PlotObservationID)'

ggsave(here("output", "figures", "manuscript", "appendix2.jpeg"),
  dpi = 300, width = 4, height = 5)

```

Fit RF models (level 1)

Without refinement

N points per category

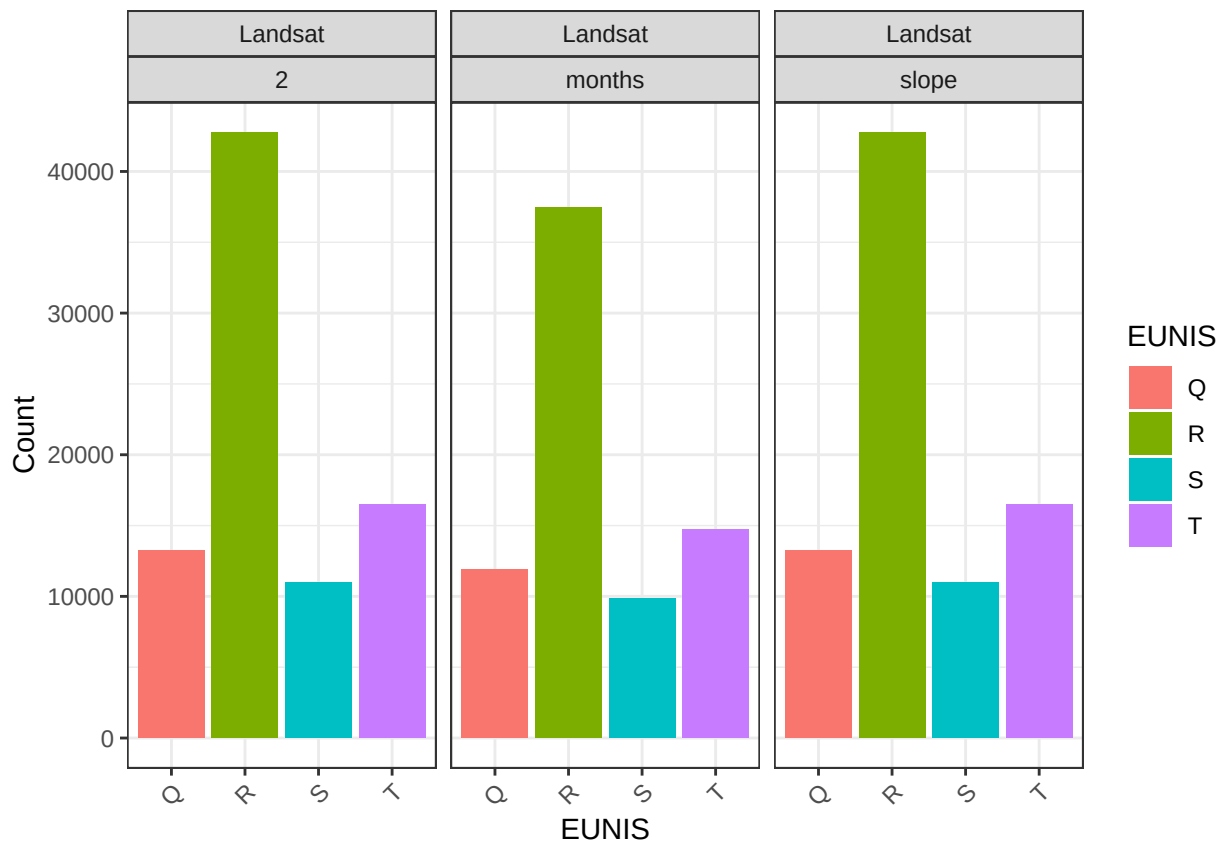
```

bind_rows(
  filtered_data_slope_Landsat %>% count(EUNIS1) %>%
    pivot_wider(names_from = EUNIS1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "slope") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_months_Landsat %>% count(EUNIS1) %>%
    pivot_wider(names_from = EUNIS1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "months") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_2_Landsat %>% count(EUNIS1) %>%
    pivot_wider(names_from = EUNIS1, values_from = n) %>%
    mutate(sensor = "S2", method = "2")
)

```

```
## # A tibble: 3 x 6
##   sensor method      Q      R      S      T
##   <chr>   <chr> <int> <int> <int> <int>
## 1 Landsat slope  13255 42738 11000 16504
## 2 Landsat months 11927 37477  9876 14717
## 3 S2      2      13255 42739 11000 16504
```

```
bind_rows(
  filtered_data_slope_Landsat %>% count(EUNIS1) %>%
    pivot_wider(names_from = EUNIS1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "slope") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_months_Landsat %>% count(EUNIS1) %>%
    pivot_wider(names_from = EUNIS1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "months") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_2_Landsat %>% count(EUNIS1) %>%
    pivot_wider(names_from = EUNIS1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "2") %>%
    select(sensor, method, Q, R, S, T)
) %>%
  pivot_longer(cols = 3:6, names_to = "EUNIS", values_to = "Count") %>%
  ggplot(aes(x = EUNIS, y = Count, fill = EUNIS, group = EUNIS)) +
  geom_col() +
  facet_wrap(~ sensor + method) + theme_bw() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Split into training and test data sets

Split data:

```
# Slope
set.seed(123)
split_data_slope_Landsat <- split_train_test(filtered_data_slope_Landsat)

train_data_slope_Landsat <- split_data_slope_Landsat$train
test_data_slope_Landsat <- split_data_slope_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_slope_Landsat$PlotObservationID %in% test_data_slope_Landsat$PlotObservationID)

## [1] 0

# Months
set.seed(123)
split_data_months_Landsat <- split_train_test(filtered_data_months_Landsat)

train_data_months_Landsat <- split_data_months_Landsat$train
test_data_months_Landsat <- split_data_months_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_months_Landsat$PlotObservationID %in% test_data_months_Landsat$PlotObservationID)

## [1] 0

# 2
set.seed(123)
split_data_2_Landsat <- split_train_test(filtered_data_2_Landsat)

train_data_2_Landsat <- split_data_2_Landsat$train
test_data_2_Landsat <- split_data_2_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_2_Landsat$PlotObservationID %in% test_data_2_Landsat$PlotObservationID)

## [1] 0
```

Fit models

```
models_no_refin <- c("rf_slope_Landsat", "rf_months_Landsat", "rf_2_Landsat")
```

Print the time:

```
for (model_name in models_no_refin) {
  obj <- get(model_name)
  cat("Time for", model_name, ":\n")
  print(obj$time)
}
```

```
## Time for rf_slope_Landsat :
##   user  system elapsed
##  59.18   48.06 1157.19
## Time for rf_months_Landsat :
##   user  system elapsed
##  53.33   17.37  800.64
## Time for rf_2_Landsat :
##   user  system elapsed
##  57.78   18.11  859.62
```

Save:

Predictions

```
predictions_no_refin <- c("rf_slope_Landsat_prediction",
                          "rf_months_Landsat_prediction",
                          "rf_2_Landsat_prediction")
```

Save:

Confusion matrices

```
for (prediction_name in predictions_no_refin) {
  # Derive test data name by replacing "_prediction" with ""
  # and "rf_" with "test_data_"
  test_data_name <- paste0("test_data_", sub("^rf_", "", sub("_prediction$", "",
                                                                prediction_name)))

  # Get prediction and test data
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Compute confusion matrix
  cat("Confusion Matrix for", sub("_prediction$", "", prediction_name), ":\n")
  print(confusionMatrix(prediction, test_data$EUNIS1))
  cat("\n")
}
```

```
## Confusion Matrix for rf_slope_Landsat :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q     R     S     T
##           Q   325   223   163    30
##           R  2735 11089  1943   816
##           S   520   654  1067     3
##           T   370   895   118  4099
##
## Overall Statistics
```

```

##
##          Accuracy : 0.6619
##          95% CI : (0.656, 0.6677)
##    No Information Rate : 0.5134
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.4369
##
##    McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##          Class: Q Class: R Class: S Class: T
## Sensitivity      0.08228  0.8622  0.32422  0.8284
## Specificity      0.98028  0.5493  0.94591  0.9312
## Pos Pred Value   0.43860  0.6687  0.47549  0.7477
## Neg Pred Value   0.85088  0.7907  0.90248  0.9566
## Prevalence       0.15768  0.5134  0.13138  0.1975
## Detection Rate   0.01297  0.4427  0.04259  0.1636
## Detection Prevalence 0.02958  0.6620  0.08958  0.2188
## Balanced Accuracy 0.53128  0.7057  0.63506  0.8798
##
## Confusion Matrix for rf_months_Landsat :
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    Q    R    S    T
##          Q  552  349  218   61
##          R 2211 9483 1532  605
##          S  530  646 1080    4
##          T  334  838  103 3654
##
## Overall Statistics
##
##          Accuracy : 0.6653
##          95% CI : (0.659, 0.6715)
##    No Information Rate : 0.5097
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.4575
##
##    McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##          Class: Q Class: R Class: S Class: T
## Sensitivity      0.15219  0.8380  0.36822  0.8451
## Specificity      0.96619  0.6005  0.93876  0.9287
## Pos Pred Value   0.46780  0.6856  0.47788  0.7413
## Neg Pred Value   0.85371  0.7810  0.90707  0.9612
## Prevalence       0.16338  0.5097  0.13212  0.1948
## Detection Rate   0.02486  0.4272  0.04865  0.1646
## Detection Prevalence 0.05315  0.6230  0.10180  0.2220
## Balanced Accuracy 0.55919  0.7193  0.65349  0.8869

```

```
##
## Confusion Matrix for rf_2_Landsat :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q     R     S     T
##           Q   121    55    49    4
##           R  3049 11434  2227   849
##           S   460   557   914    0
##           T   348   790   119  4074
##
## Overall Statistics
##
##           Accuracy : 0.6604
##           95% CI : (0.6545, 0.6663)
##           No Information Rate : 0.5124
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4218
##
## Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: Q Class: R Class: S Class: T
## Sensitivity      0.030417  0.8908  0.27622  0.8269
## Specificity      0.994875  0.4985  0.95322  0.9375
## Pos Pred Value   0.528384  0.6512  0.47333  0.7642
## Neg Pred Value   0.844607  0.8128  0.89641  0.9567
## Prevalence       0.158802  0.5124  0.13210  0.1967
## Detection Rate   0.004830  0.4564  0.03649  0.1626
## Detection Prevalence 0.009142  0.7010  0.07709  0.2128
## Balanced Accuracy 0.512646  0.6947  0.61472  0.8822
```

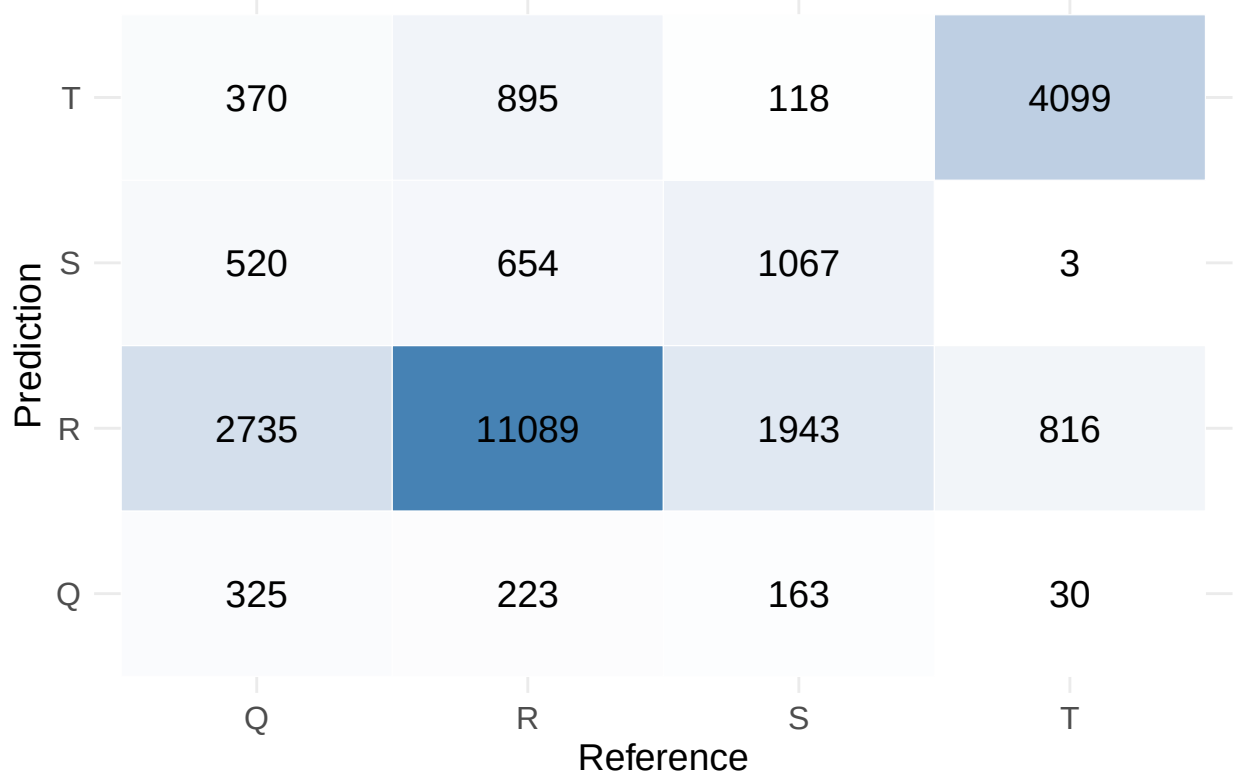
```
for (prediction_name in predictions_no_refin) {
  # Derive name of test data
  test_data_name <- paste0("test_data_", sub("^rf_", "", sub("_prediction$", "",
    prediction_name)))

  # Obtain objects
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Personalized title
  title <- paste("#Confusion Matrix for",
    sub("_prediction$", "",
      prediction_name))

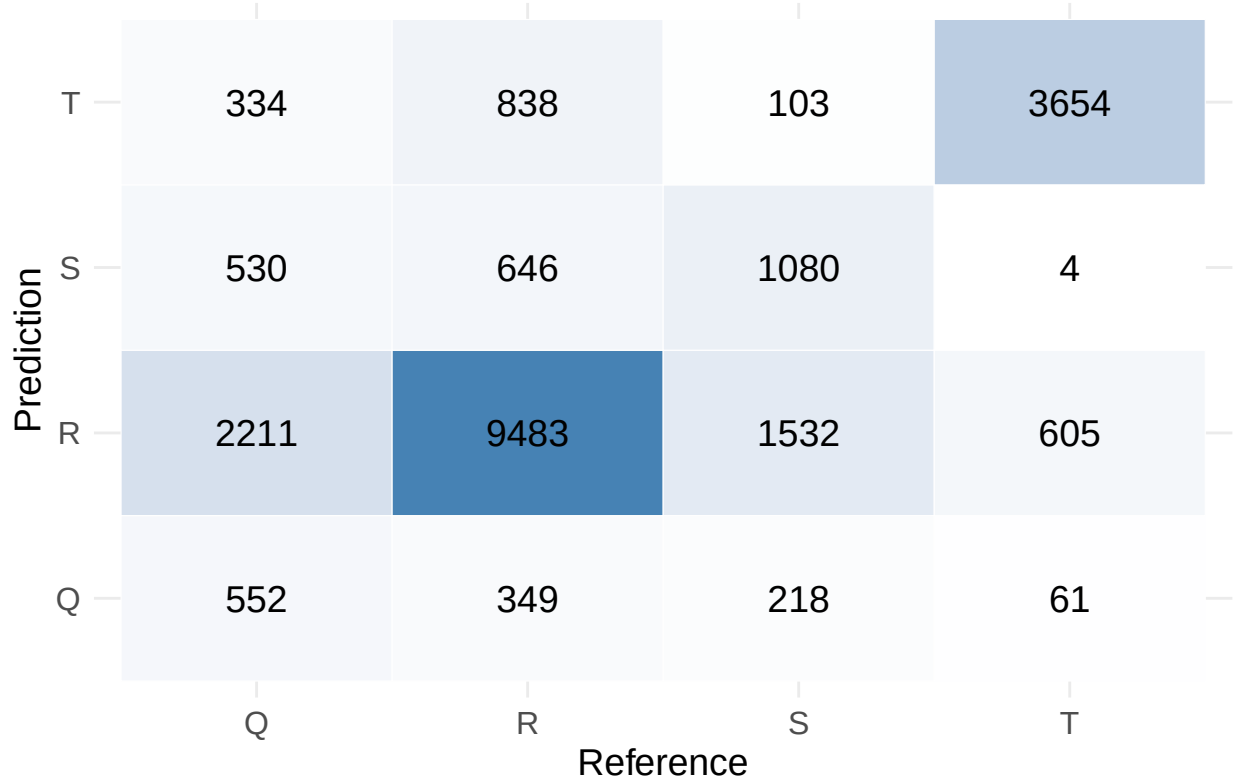
  # Plot
  print(plot_confusion_matrix(prediction, test_data$EUNIS1, title))
}
```

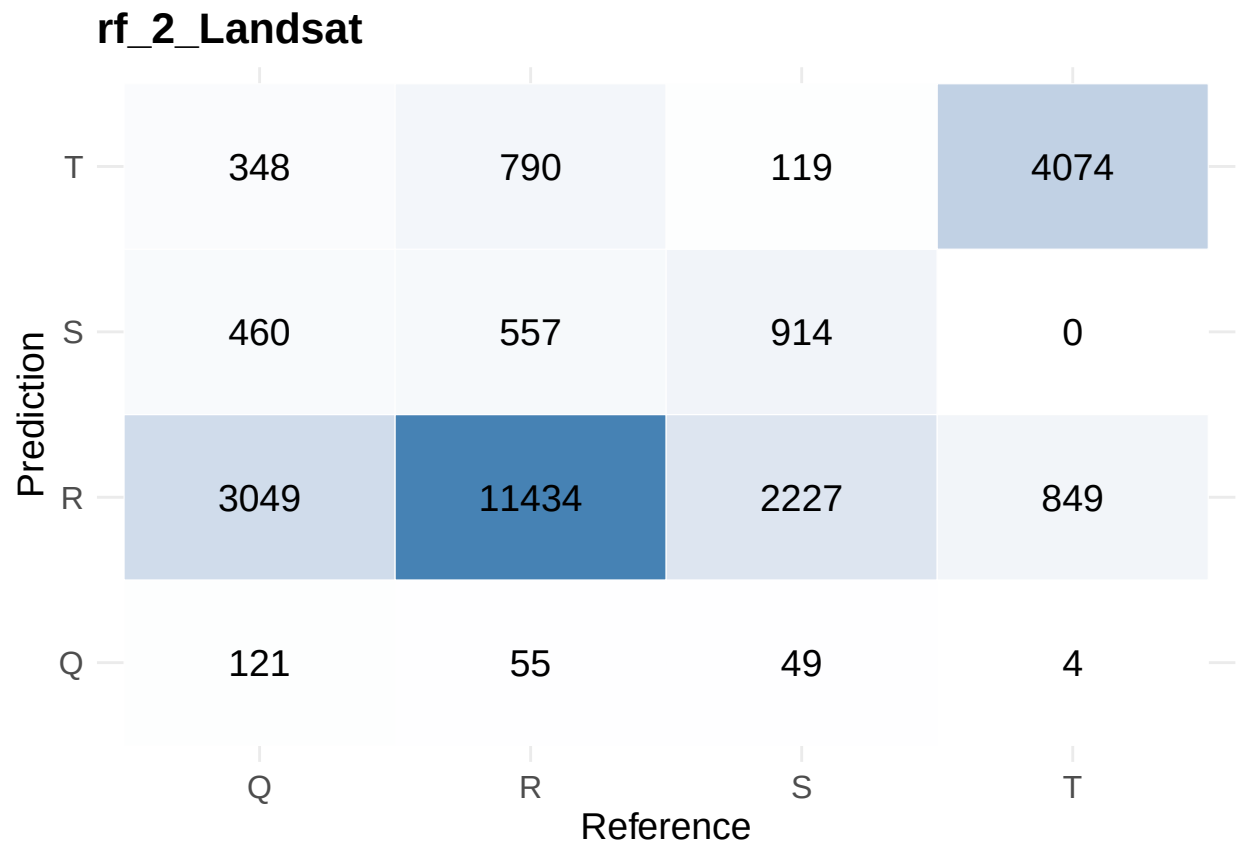
rf_slope_Landsat



Plots

rf_months_Landsat





Refinement step 1

Based on PAs “Morralla protocol”.

Compute predictions for all data and combine with original data

```
filtered_data_with_probs <- c("filtered_data_slope_Landsat_with_probs",
                              "filtered_data_months_Landsat_with_probs",
                              "filtered_data_2_Landsat_with_probs")
```

Save:

Detect probability columns

```
prob_cols_slope_Landsat <- names(filtered_data_slope_Landsat_with_probs)[grepl("^prob_", names(filtered_data_slope_Landsat_with_probs))]
prob_cols_months_Landsat <- names(filtered_data_months_Landsat_with_probs)[grepl("^prob_", names(filtered_data_months_Landsat_with_probs))]
prob_cols_2_Landsat <- names(filtered_data_2_Landsat_with_probs)[grepl("^prob_", names(filtered_data_2_Landsat_with_probs))]
```

Calculate auxiliary metrics

```
df_aux_slope_Landsat <- calculate_df_aux(filtered_data_slope_Landsat_with_probs,
                                          prob_cols_slope_Landsat)
df_aux_months_Landsat <- calculate_df_aux(filtered_data_months_Landsat_with_probs,
                                          prob_cols_months_Landsat)
df_aux_2_Landsat <- calculate_df_aux(filtered_data_2_Landsat_with_probs,
                                     prob_cols_2_Landsat)
```

Apply filtering function

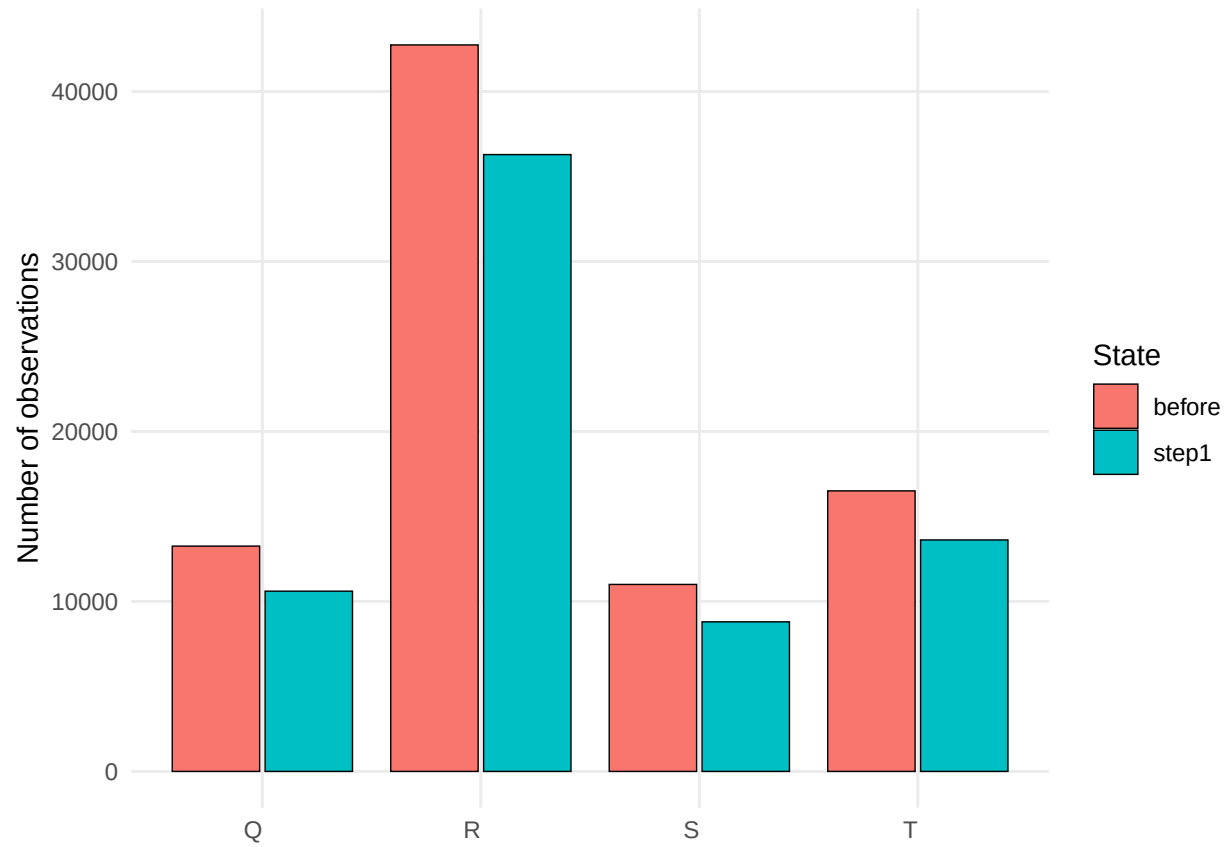
Applying “Morralla” protocol once, removing negatives until 20% of the class and, if there is room, uncertain positives until 10% of the rest.

```
df_final_20_10_slope_Landsat <- df_aux_slope_Landsat %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_months_Landsat <- df_aux_months_Landsat %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_2_Landsat <- df_aux_2_Landsat %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
```

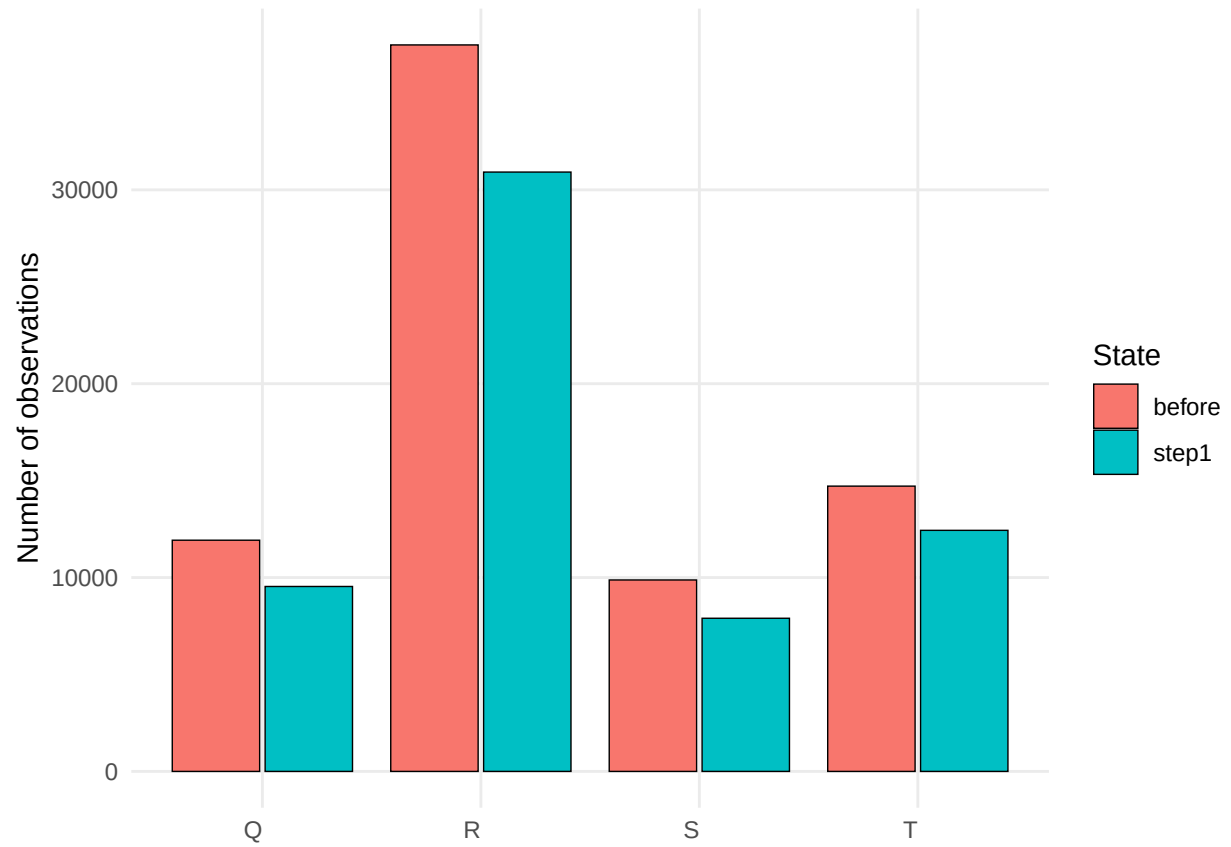
```
filtered_data_20_10_slope_Landsat_probs <- filtered_data_slope_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_slope_Landsat$PlotObservationID)
filtered_data_20_10_months_Landsat_probs <- filtered_data_months_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_months_Landsat$PlotObservationID)
filtered_data_20_10_2_Landsat_probs <- filtered_data_2_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_2_Landsat$PlotObservationID)
```

Compare classes before and after

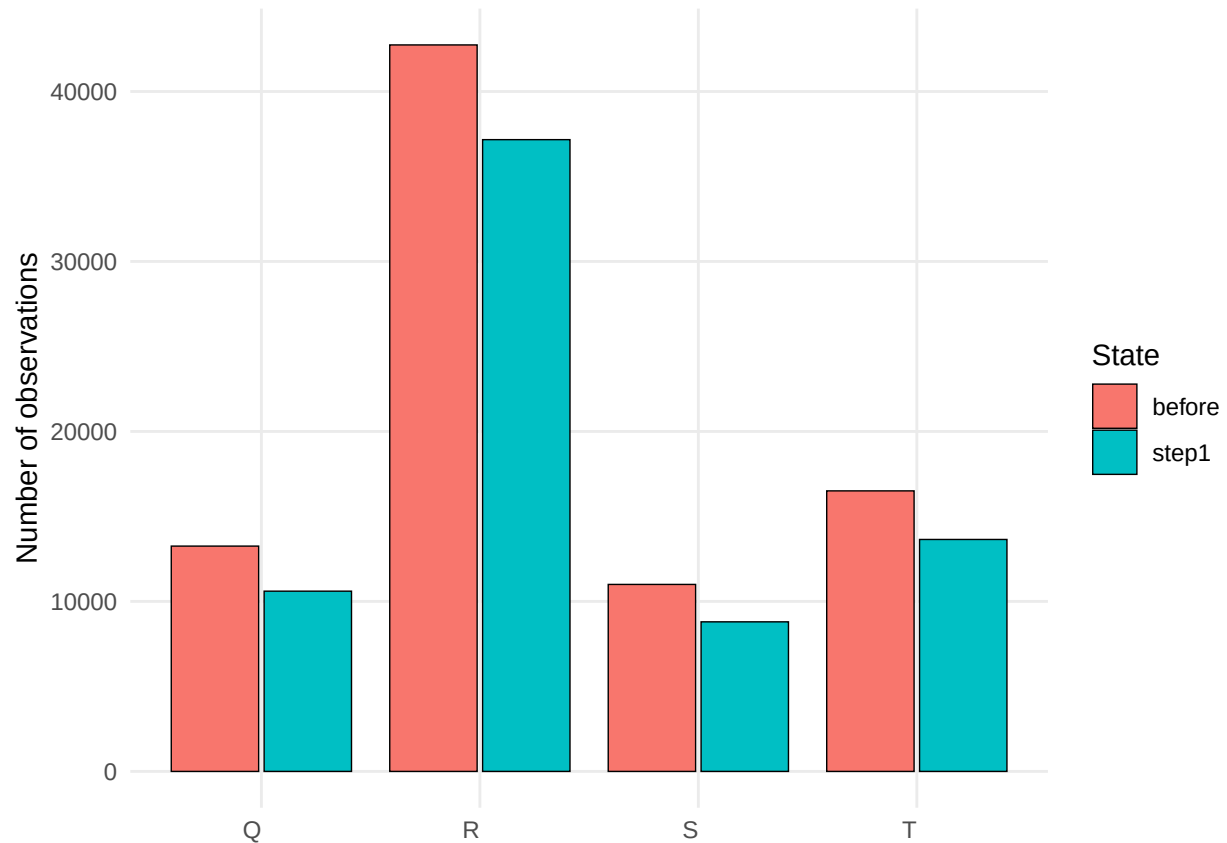
```
compare_classes(filtered_data_slope_Landsat, filtered_data_20_10_slope_Landsat_probs)
```



```
compare_classes(filtered_data_months_Landsat, filtered_data_20_10_months_Landsat_probs)
```



```
compare_classes(filtered_data_2_Landsat, filtered_data_20_10_2_Landsat_probs)
```



Split into training and testing

```
# Landsat
# Slope
set.seed(123)
split_data_20_10_slope_Landsat <- split_train_test(filtered_data_20_10_slope_Landsat_probs)

train_data_20_10_slope_Landsat <- split_data_20_10_slope_Landsat$train
test_data_20_10_slope_Landsat <- split_data_20_10_slope_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_slope_Landsat$PlotObservationID %in% test_data_20_10_slope_Landsat$PlotObservationID)

## [1] 0
```

```
# Months
set.seed(123)
split_data_20_10_months_Landsat <- split_train_test(filtered_data_20_10_months_Landsat_probs)

train_data_20_10_months_Landsat <- split_data_20_10_months_Landsat$train
test_data_20_10_months_Landsat <- split_data_20_10_months_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_months_Landsat$PlotObservationID %in% test_data_20_10_months_Landsat$PlotObservationID)
```

```
## [1] 0
```

```
# 2
set.seed(123)
split_data_20_10_2_Landsat <- split_train_test(filtered_data_20_10_2_Landsat_probs)

train_data_20_10_2_Landsat <- split_data_20_10_2_Landsat$train
test_data_20_10_2_Landsat <- split_data_20_10_2_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2_Landsat$PlotObservationID %in% test_data_20_10_2_Landsat$PlotObservationID)
```

```
## [1] 0
```

Fit models

```
models_refin_20_10 <- c("rf_slope_Landsat_refin_20_10",
                        "rf_months_Landsat_refin_20_10",
                        "rf_2_Landsat_refin_20_10")
```

Save:

Predictions

```
predictions_refin_20_10 <- c("rf_slope_Landsat_refin_20_10_prediction",
                             "rf_months_Landsat_refin_20_10_prediction",
                             "rf_2_Landsat_refin_20_10_prediction")
```

Save:

Confusion matrices

```
for (prediction_name in predictions_refin_20_10) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10$", "",
                                sub("^rf_", "test_data_20_10_",
                                    sub("_prediction$", "", prediction_name))))

  # Get prediction and test data
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Compute confusion matrix
  cat("Confusion Matrix for", sub("_prediction$", "", prediction_name), ":\n")
  print(confusionMatrix(prediction, test_data$EUNIS1))
  cat("\n")
}
```

```

## Confusion Matrix for rf_slope_Landsat_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##           Q   657   331   314   38
##           R  1531 10133   837   208
##           S   741   285  1386    0
##           T   289   206    64  3773
##
## Overall Statistics
##
##           Accuracy : 0.767
##           95% CI : (0.7612, 0.7728)
##           No Information Rate : 0.5269
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.6201
##
## McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: Q Class: R Class: S Class: T
## Sensitivity      0.20416   0.9250   0.53287   0.9388
## Specificity      0.96114   0.7382   0.94360   0.9667
## Pos Pred Value   0.49030   0.7973   0.57463   0.8710
## Neg Pred Value   0.86835   0.8983   0.93390   0.9851
## Prevalence       0.15476   0.5269   0.12509   0.1933
## Detection Rate   0.03160   0.4873   0.06666   0.1815
## Detection Prevalence 0.06444   0.6112   0.11600   0.2083
## Balanced Accuracy 0.58265   0.8316   0.73824   0.9527
##
## Confusion Matrix for rf_months_Landsat_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##           Q   929   352   376   68
##           R  1056  8600   572   153
##           S   631   229  1367    0
##           T   232   167    26  3481
##
## Overall Statistics
##
##           Accuracy : 0.7883
##           95% CI : (0.7823, 0.7942)
##           No Information Rate : 0.5125
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.6662
##
## McNemar's Test P-Value : < 2.2e-16
##

```

```
## Statistics by Class:
##
##
##          Class: Q Class: R Class: S Class: T
## Sensitivity      0.32619   0.9200   0.58394   0.9403
## Specificity      0.94828   0.7997   0.94591   0.9708
## Pos Pred Value   0.53855   0.8284   0.61383   0.8912
## Neg Pred Value   0.88380   0.9048   0.93917   0.9846
## Prevalence       0.15615   0.5125   0.12835   0.2030
## Detection Rate   0.05093   0.4715   0.07495   0.1909
## Detection Prevalence 0.09458   0.5692   0.12210   0.2142
## Balanced Accuracy 0.63724   0.8598   0.76492   0.9555
##
## Confusion Matrix for rf_2_Landsat_refin_20_10 :
## Confusion Matrix and Statistics
##
##          Reference
## Prediction      Q      R      S      T
##          Q    339    146    157    13
##          R   2033   10660   1142    211
##          S    662    244   1248     0
##          T    221    168     48   3774
##
## Overall Statistics
##
##          Accuracy : 0.7605
##          95% CI : (0.7547, 0.7663)
##          No Information Rate : 0.5325
##          P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.5938
##
## Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##          Class: Q Class: R Class: S Class: T
## Sensitivity      0.10415   0.9503   0.48092   0.9440
## Specificity      0.98226   0.6562   0.95095   0.9744
## Pos Pred Value   0.51756   0.7589   0.57939   0.8962
## Neg Pred Value   0.85714   0.9205   0.92878   0.9867
## Prevalence       0.15451   0.5325   0.12318   0.1898
## Detection Rate   0.01609   0.5060   0.05924   0.1792
## Detection Prevalence 0.03109   0.6668   0.10225   0.1999
## Balanced Accuracy 0.54320   0.8032   0.71594   0.9592
```

```
for (prediction_name in predictions_refin_20_10) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10$", "",
                                sub("^rf_", "test_data_20_10_",
                                    sub("_prediction$", "", prediction_name))))
}
```

```

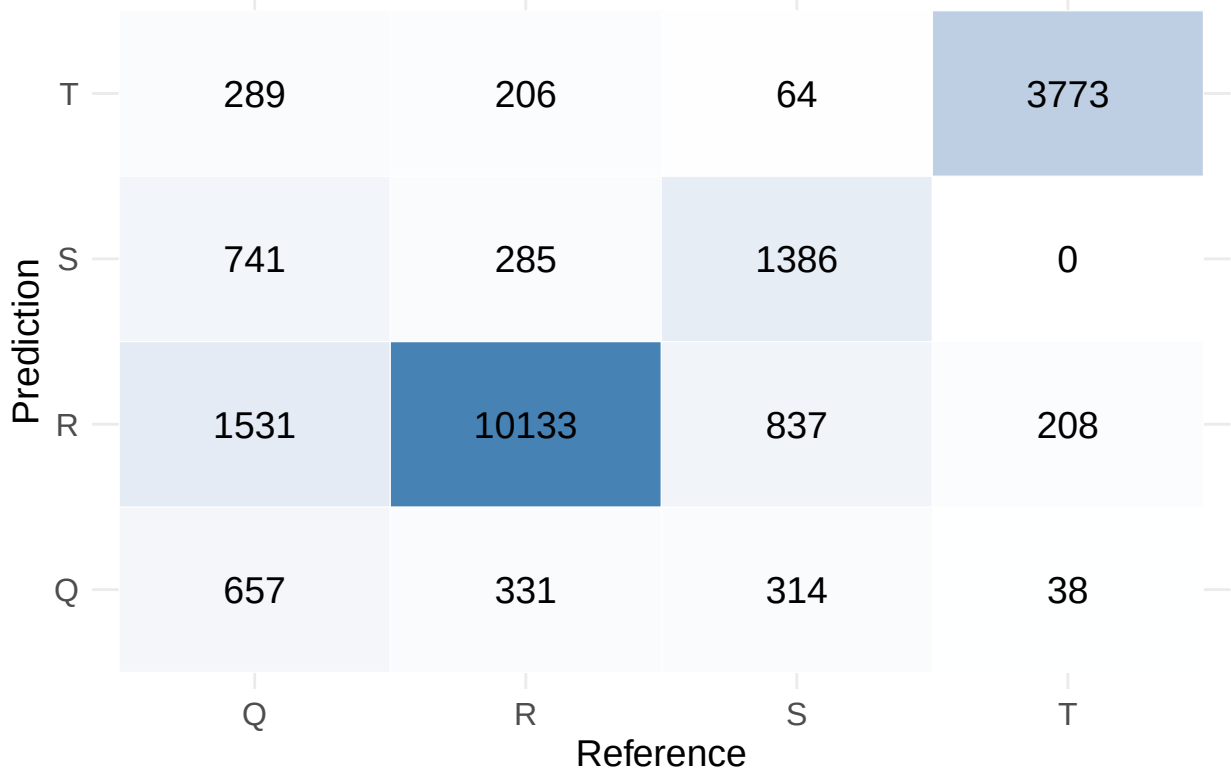
# Obtain objects
prediction <- get(prediction_name)
test_data <- get(test_data_name)

# Personalized title
title <- paste("Confusion Matrix for", sub("_prediction$", "",
                                           prediction_name))

# Plot
print(plot_confusion_matrix(prediction, test_data$EUNIS1, title))
}

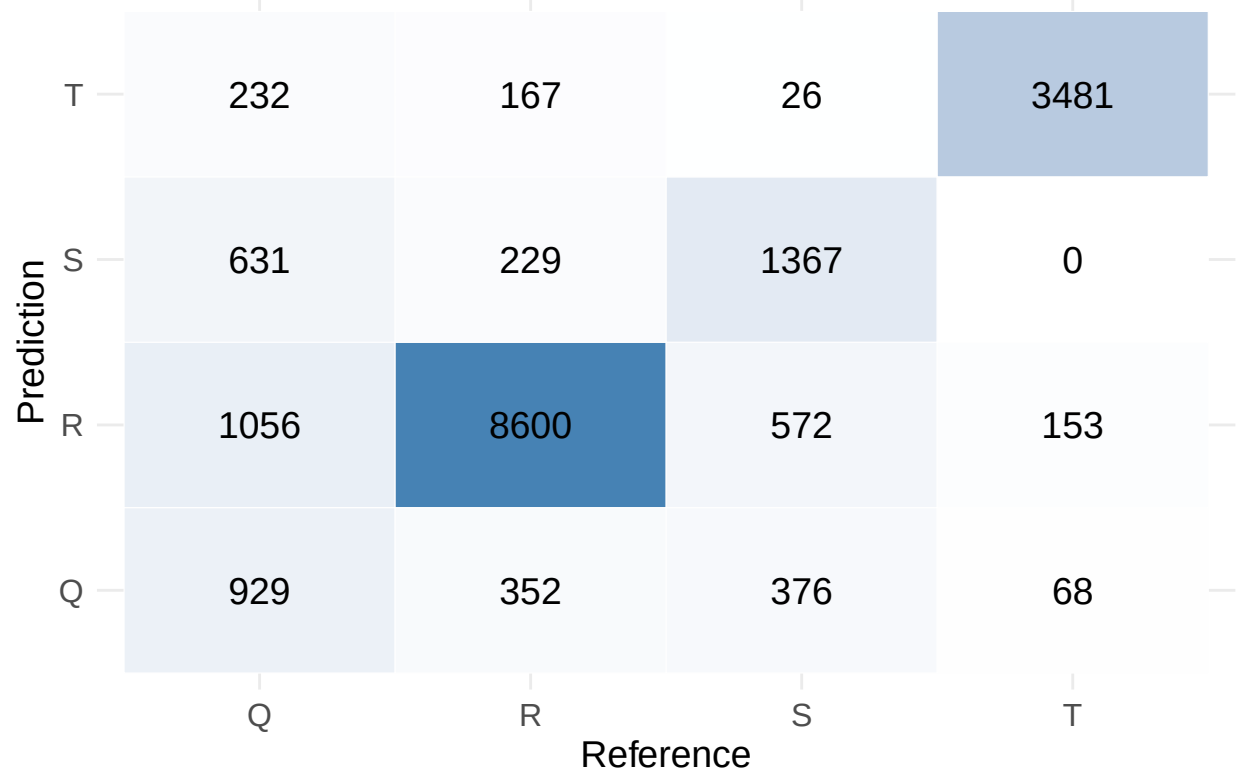
```

Confusion Matrix for rf_slope_Landsat_refin_20_10



Plots

Confusion Matrix for rf_months_Landsat_refin_20_10



Confusion Matrix for rf_2_Landsat_refin_20_10

Prediction	T	221	168	48	3774
	S	662	244	1248	0
	R	2033	10660	1142	211
	Q	339	146	157	13
		Q	R	S	T
		Reference			

Refinement step 2

Then refitting the model and applying “Morralla” protocol again, removing only negatives until 20% of the class.

```
filtered_data_with_probs_2x <- c("filtered_data_slope_Landsat_with_probs_2x",
                                "filtered_data_months_Landsat_with_probs_2x",
                                "filtered_data_2_Landsat_with_probs_2x")
```

Compute predictions for all data and combine with original data Save:

```
df_aux_slope_Landsat_2x <- calculate_df_aux(filtered_data_slope_Landsat_with_probs_2x,
                                             prob_cols_slope_Landsat)
df_aux_months_Landsat_2x <- calculate_df_aux(filtered_data_months_Landsat_with_probs_2x,
                                              prob_cols_months_Landsat)
df_aux_2_Landsat_2x <- calculate_df_aux(filtered_data_2_Landsat_with_probs_2x,
                                         prob_cols_2_Landsat)
```

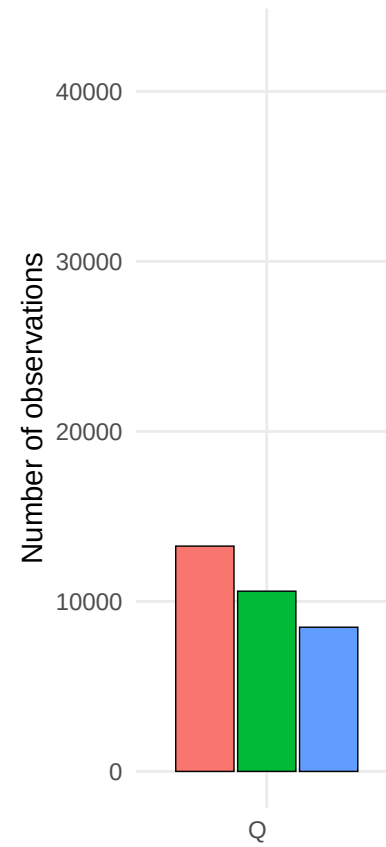
Calculate auxiliary metrics

```
df_final_20_10_slope_Landsat_2x <- df_aux_slope_Landsat_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_months_Landsat_2x <- df_aux_months_Landsat_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_2_Landsat_2x <- df_aux_2_Landsat_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
```

```
filtered_data_20_10_slope_Landsat_probs_2x <- filtered_data_slope_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_slope_Landsat_2x$PlotObservationID)
filtered_data_20_10_months_Landsat_probs_2x <- filtered_data_months_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_months_Landsat_2x$PlotObservationID)
filtered_data_20_10_2_Landsat_probs_2x <- filtered_data_2_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_2_Landsat_2x$PlotObservationID)
```

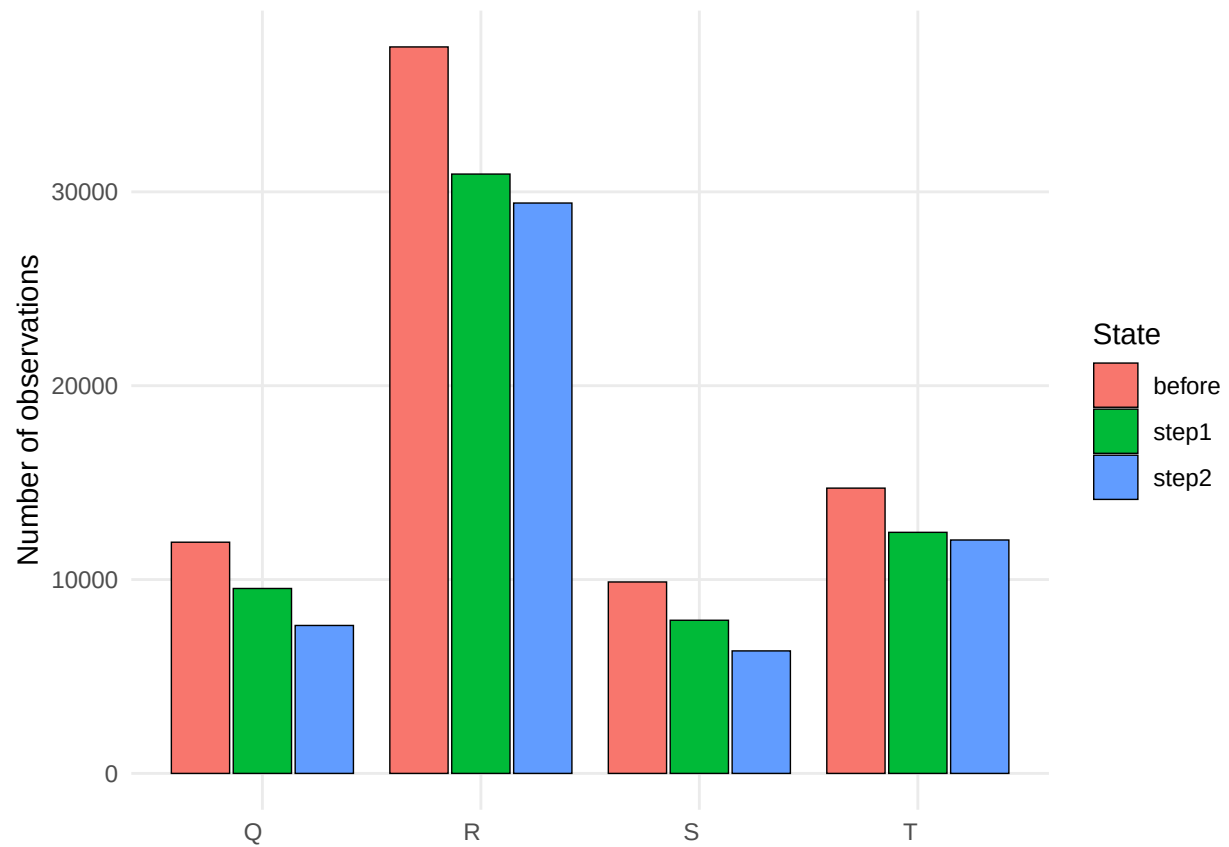
Apply filtering function

```
compare_classes(filtered_data_slope_Landsat,
  filtered_data_20_10_slope_Landsat_probs,
  filtered_data_20_10_slope_Landsat_probs_2x)
```

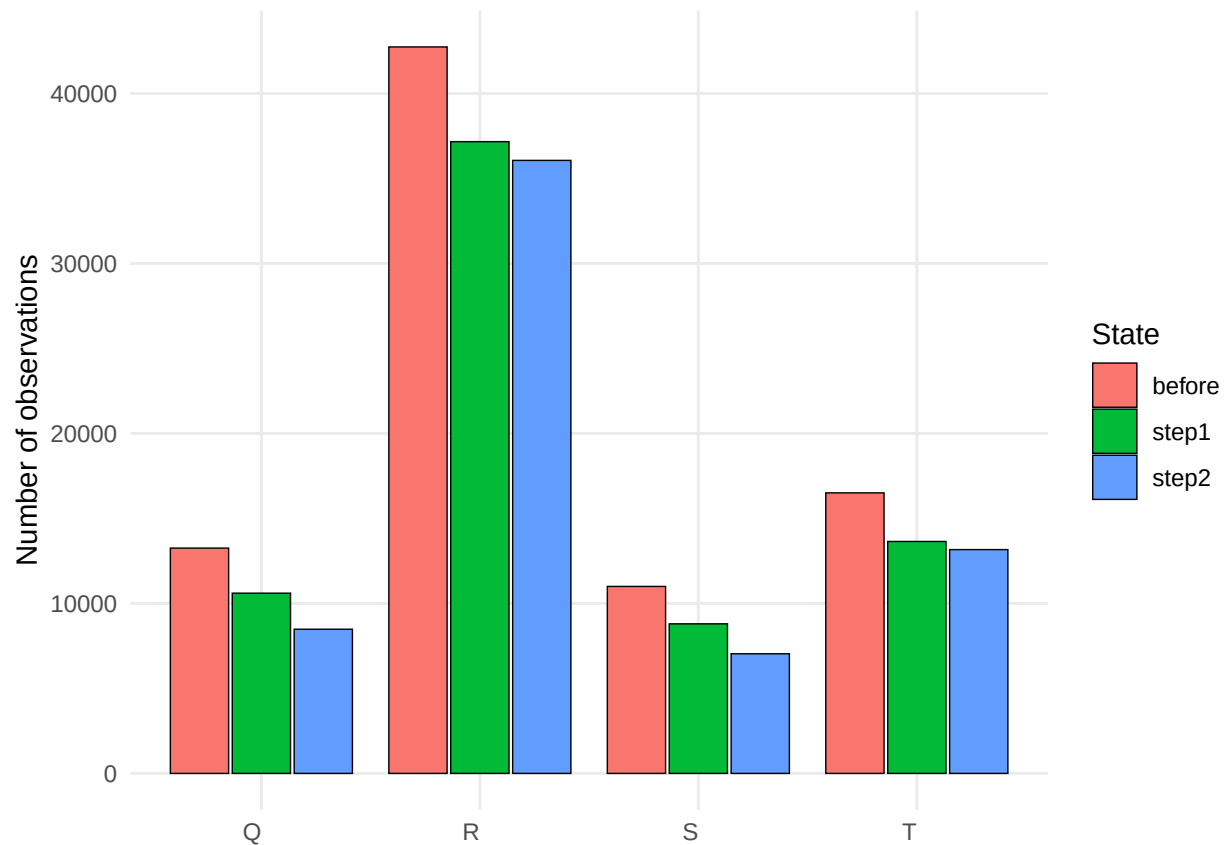


SEE IF ADDING METHOD 2: (Appendix 3) Compare classes before and after

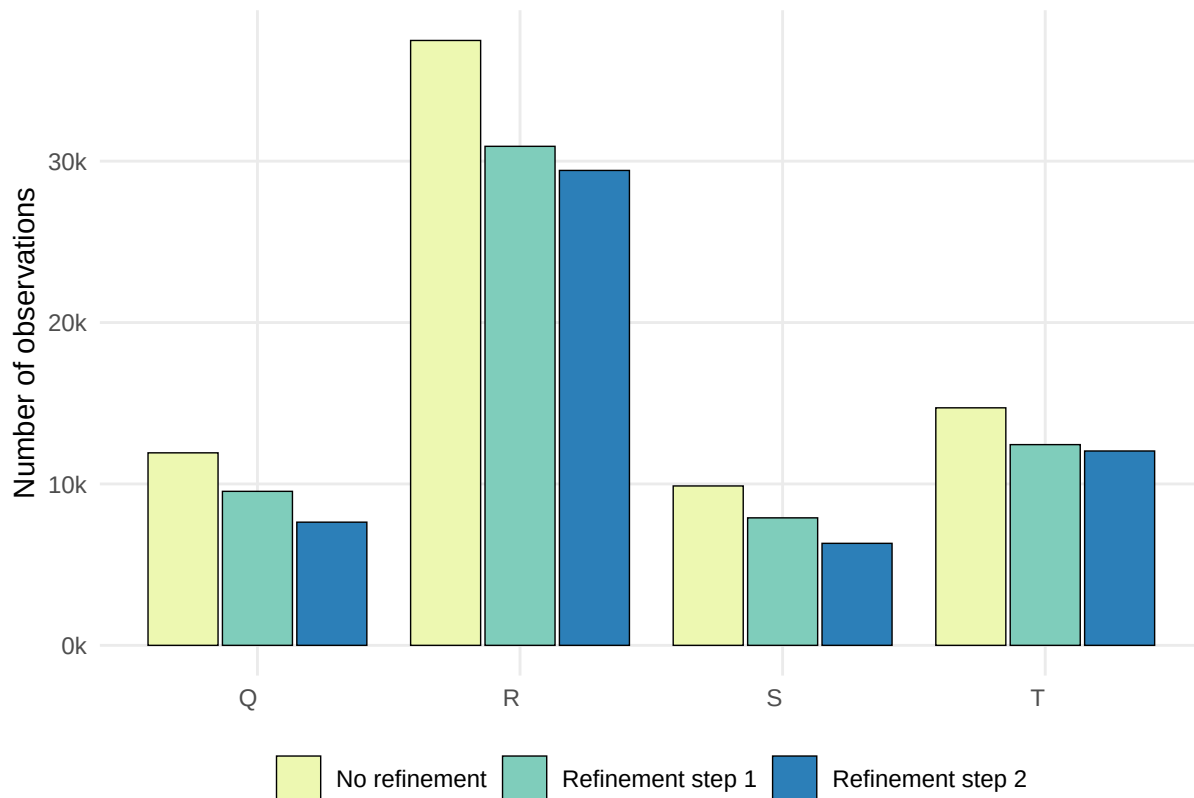
```
compare_classes(filtered_data_months_Landsat,  
                filtered_data_20_10_months_Landsat_probs,  
                filtered_data_20_10_months_Landsat_probs_2x)
```



```
compare_classes(filtered_data_2_Landsat,  
                filtered_data_20_10_2_Landsat_probs,  
                filtered_data_20_10_2_Landsat_probs_2x)
```



```
compare_classes(filtered_data_months_Landsat,
                filtered_data_20_10_months_Landsat_probs,
                filtered_data_20_10_months_Landsat_probs_2x,
                return = "plot") +
plot_layout(guides = "collect") &
scale_fill_manual(
  breaks = c("before", "step1", "step2"),
  labels = c("No refinement", "Refinement step 1", "Refinement step 2"),
  values = c(
    "before" = "#edf8b1",
    "step1" = "#7fcdbb",
    "step2" = "#2c7fb8"
  )
) &
scale_y_continuous(labels = label_number(scale = 1e-3, suffix = "k")) &
theme(legend.position = "bottom", legend.title = element_blank())
```



```
ggsave(here("output", "figures", "manuscript", "appendix3.jpeg"),
       dpi = 300, width = 5, height = 4)
```

```
# Landsat
# Slope
set.seed(123)
split_data_20_10_2x_slope_Landsat <- split_train_test(filtered_data_20_10_slope_Landsat_probs_2x)

train_data_20_10_2x_slope_Landsat <- split_data_20_10_2x_slope_Landsat$train
test_data_20_10_2x_slope_Landsat <- split_data_20_10_2x_slope_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_slope_Landsat$PlotObservationID %in% test_data_20_10_2x_slope_Landsat$PlotObservationID)
```

Split into training and testing

```
## [1] 0
```

```
# Months
set.seed(123)
split_data_20_10_2x_months_Landsat <- split_train_test(filtered_data_20_10_months_Landsat_probs_2x)
```

```

train_data_20_10_2x_months_Landsat <- split_data_20_10_2x_months_Landsat$train
test_data_20_10_2x_months_Landsat <- split_data_20_10_2x_months_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_months_Landsat$PlotObservationID %in% test_data_20_10_2x_months_Landsat$PlotObs

## [1] 0

```

```

# 2
set.seed(123)
split_data_20_10_2x_2_Landsat <- split_train_test(filtered_data_20_10_2_Landsat_probs_2x)

train_data_20_10_2x_2_Landsat <- split_data_20_10_2x_2_Landsat$train
test_data_20_10_2x_2_Landsat <- split_data_20_10_2x_2_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_2_Landsat$PlotObservationID %in% test_data_20_10_2x_2_Landsat$PlotObservationID

## [1] 0

```

```

models_refin_20_10_2x <- c("rf_slope_Landsat_refin_20_10_2x",
                          "rf_months_Landsat_refin_20_10_2x",
                          "rf_2_Landsat_refin_20_10_2x")

```

Fit models Save:

```

predictions_refin_20_10_2x <- c("rf_slope_Landsat_refin_20_10_2x_prediction",
                              "rf_months_Landsat_refin_20_10_2x_prediction",
                              "rf_2_Landsat_refin_20_10_2x_prediction")

```

Predictions Save:

```

for (prediction_name in predictions_refin_20_10_2x) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10_2x$", "",
                              sub("^rf_", "test_data_20_10_2x_",
                              sub("_prediction$", "", prediction_name))))

  # Get prediction and test data
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Compute confusion matrix

```

```

cat("Confusion Matrix for", sub("_prediction$", "", prediction_name), ":\n")
print(confusionMatrix(prediction, test_data$EUNIS1))
cat("\n")
}

```

Confusion matrices

```

## Confusion Matrix for rf_slope_Landsat_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   Q    R    S    T
##           Q  738  275  344   42
##           R 1090 9808  407  160
##           S  552  153 1377    0
##           T  168  116   11 3779
##
## Overall Statistics
##
##           Accuracy : 0.8256
##           95% CI : (0.8201, 0.8309)
##           No Information Rate : 0.5443
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7116
##
## Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: Q Class: R Class: S Class: T
## Sensitivity         0.28964   0.9474   0.6438   0.9493
## Specificity         0.95987   0.8088   0.9582   0.9804
## Pos Pred Value      0.52752   0.8555   0.6614   0.9276
## Neg Pred Value      0.89728   0.9280   0.9550   0.9865
## Prevalence          0.13396   0.5443   0.1125   0.2093
## Detection Rate      0.03880   0.5157   0.0724   0.1987
## Detection Prevalence 0.07355   0.6028   0.1095   0.2142
## Balanced Accuracy    0.62476   0.8781   0.8010   0.9648
##
## Confusion Matrix for rf_months_Landsat_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   Q    R    S    T
##           Q 1044  215  267   46
##           R  662 8348  224  131
##           S  521  134 1317    0
##           T  152  109   4 3452
##
## Overall Statistics
##
##           Accuracy : 0.8517

```

```

##          95% CI : (0.8462, 0.8571)
##    No Information Rate : 0.5297
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.7633
##
##    McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##          Class: Q Class: R Class: S Class: T
## Sensitivity      0.43884  0.9480  0.72682  0.9512
## Specificity      0.96294  0.8699  0.95579  0.9796
## Pos Pred Value   0.66412  0.8914  0.66785  0.9287
## Neg Pred Value   0.91132  0.9369  0.96622  0.9863
## Prevalence       0.14309  0.5297  0.10899  0.2183
## Detection Rate   0.06279  0.5021  0.07921  0.2076
## Detection Prevalence 0.09455  0.5633  0.11861  0.2236
## Balanced Accuracy 0.70089  0.9090  0.84130  0.9654
##
## Confusion Matrix for rf_2_Landsat_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    Q      R      S      T
##          Q   432   147   199   12
##          R 1447 10329   609   169
##          S   515   189  1295    0
##          T   174   141    18  3751
##
## Overall Statistics
##
##          Accuracy : 0.8137
##          95% CI : (0.8081, 0.8191)
##    No Information Rate : 0.5562
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.6795
##
##    McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##          Class: Q Class: R Class: S Class: T
## Sensitivity      0.16822  0.9559  0.61056  0.9540
## Specificity      0.97877  0.7419  0.95932  0.9785
## Pos Pred Value   0.54684  0.8228  0.64782  0.9185
## Neg Pred Value   0.88539  0.9306  0.95261  0.9882
## Prevalence       0.13219  0.5562  0.10918  0.2024
## Detection Rate   0.02224  0.5317  0.06666  0.1931
## Detection Prevalence 0.04067  0.6462  0.10290  0.2102
## Balanced Accuracy 0.57349  0.8489  0.78494  0.9662

```

```

for (prediction_name in predictions_refin_20_10_2x) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10_2x$", "",
                                sub("^rf_", "test_data_20_10_2x_",
                                      sub("_prediction$", "", prediction_name))))

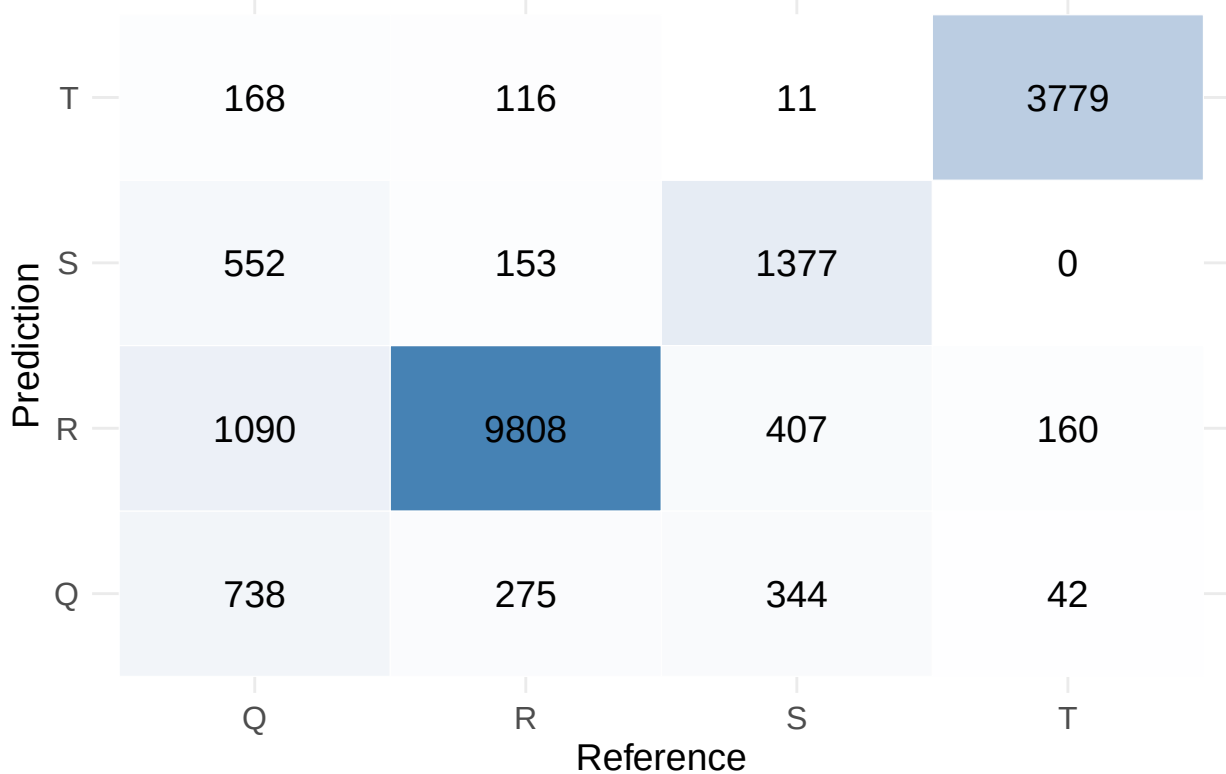
  # Obtain objects
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Personalized title
  title <- paste("Confusion Matrix for", sub("_prediction$", "",
                                              prediction_name))

  # Plot
  print(plot_confusion_matrix(prediction, test_data$EUNIS1, title))
}

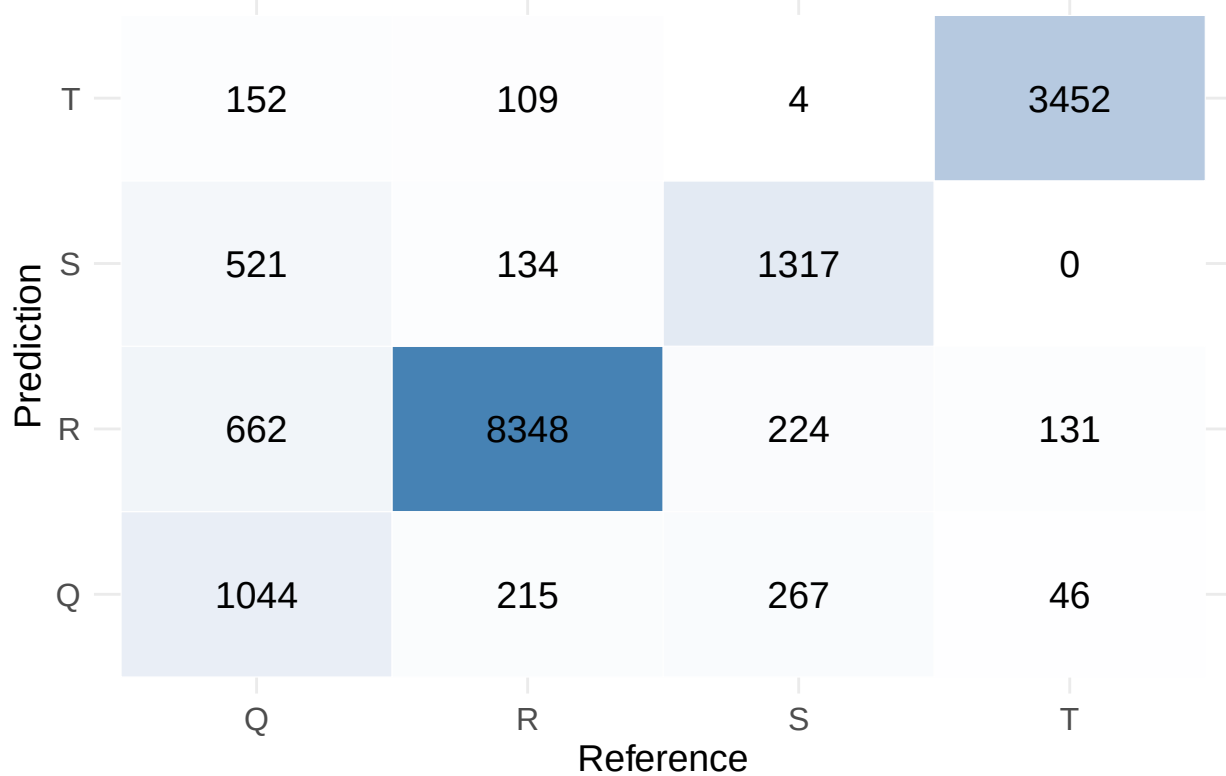
```

Confusion Matrix for rf_slope_Landsat_refin_20_10_2x

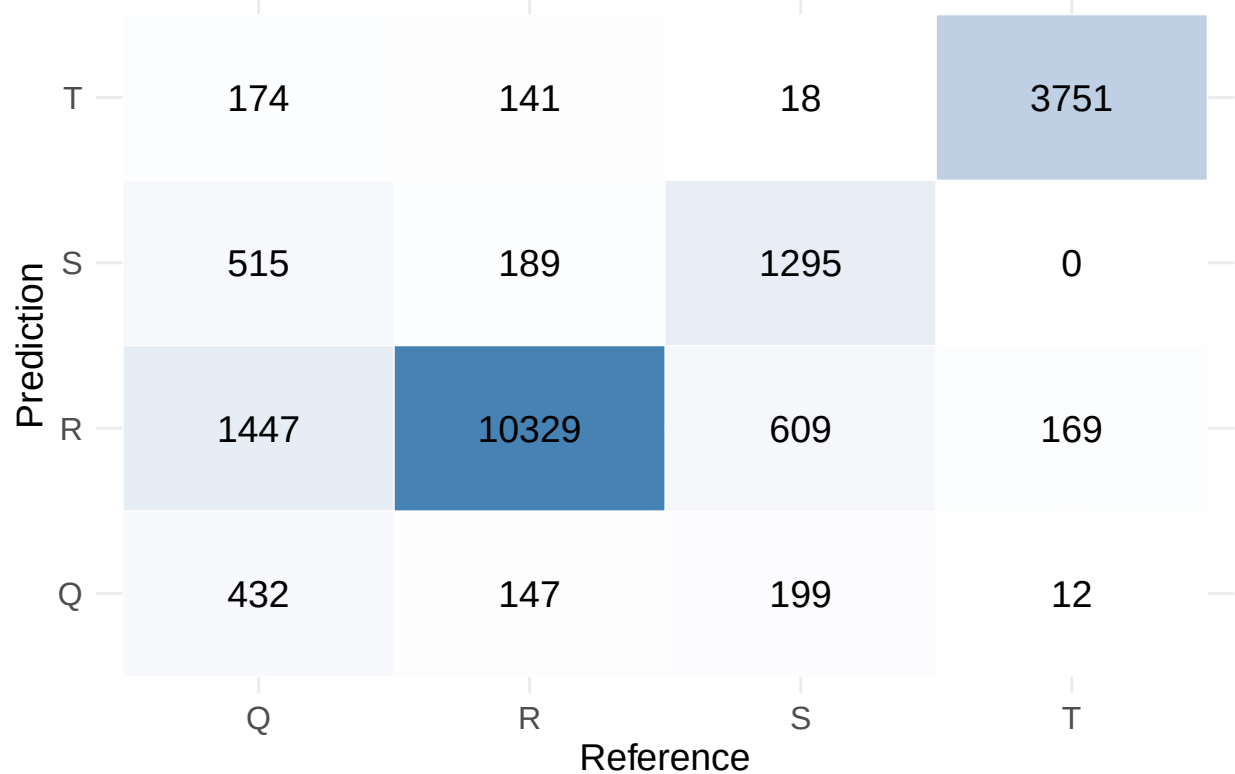


Plots

Confusion Matrix for rf_months_Landsat_refin_20_10_2x



Confusion Matrix for rf_2_Landsat_refin_20_10_2x



N obs

N obs with metrics extracted

```
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  nrow()
```

```
## [1] 120327
```

N obs after data filtering

```
# Landsat slope
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Data filtering
  # Remove rows with missing values in predictors
  dplyr::filter(if_all(all_of(vars_RF_slope_Landsat), ~ !is.na(.))) %>%
  # Remove all rows with wrong values of indices (not between -1 and 1)
  dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
  # With Landsat data, many observations removed because of high/low EVI values
```

```

# In Landsat_extraction, values of EVI outside of expected range were set as NA,
# so better not to use EVI here
# dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
  # Join ReSurvey data to get Location method
left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
# Select only variables needed
dplyr::select(PlotObservationID, EUNIS1, all_of(vars_RF_slope_Landsat),
              Lctnmth) %>%
# Keep only rows with differences > 0 in variables
filter(if_all(any_of(vars_RF_slope_Landsat[grepl("diff",
                                                    vars_RF_slope_Landsat)]),
              ~ .x > 0)) %>%
nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 117482
```

```

# Landsat months
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Data filtering
  # Remove rows with missing values in predictors
  dplyr::filter(if_all(all_of(vars_RF_months_Landsat), ~ !is.na(.))) %>%
  # Remove all rows with wrong values of indices (not between -1 and 1)
  dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
  # With Landsat data, many observations removed because of high/low EVI values
  # In Landsat_extraction, values of EVI outside of expected range were set as NA,
  # so better not to use EVI here
  # dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
  dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
  dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
  dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Select only variables needed
  dplyr::select(PlotObservationID, EUNIS1, all_of(vars_RF_months_Landsat),
                Lctnmth) %>%
  # Keep only rows with differences > 0 in variables
  filter(if_all(any_of(vars_RF_months_Landsat[grepl("diff",
                                                    vars_RF_months_Landsat)]),
                ~ .x > 0)) %>%
  nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 101000
```

```

# Landsat 2
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Data filtering
  # Remove rows with missing values in predictors
  dplyr::filter(if_all(all_of(vars_RF_2_Landsat), ~ !is.na(.))) %>%
  # Remove all rows with wrong values of indices (not between -1 and 1)
  dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
  # With Landsat data, many observations removed because of high/low EVI values
  # In Landsat_extraction, values of EVI outside of expected range were set as NA,
  # so better not to use EVI here
  # dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
  dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
  dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
  dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Select only variables needed
  dplyr::select(PlotObservationID, EUNIS1, all_of(vars_RF_2_Landsat),
                Lctnmth) %>%
  # Keep only rows with differences > 0 in variables
  filter(if_all(any_of(vars_RF_2_Landsat[grepl("diff",
                                                vars_RF_2_Landsat)]),
                ~ .x > 0)) %>%
  nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 117484
```

N obs after rough validation

```

# Landsat slope
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Data filtering
  # Remove rows with missing values in predictors
  dplyr::filter(if_all(all_of(vars_RF_slope_Landsat), ~ !is.na(.))) %>%
  # Remove all rows with wrong values of indices (not between -1 and 1)
  dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
  # With Landsat data, many observations removed because of high/low EVI values
  # In Landsat_extraction, values of EVI outside of expected range were set as NA,
  # so better not to use EVI here
  # dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
  dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
  dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
  dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Select only variables needed
  dplyr::select(PlotObservationID, EUNIS1, all_of(vars_RF_slope_Landsat),

```

```

      Lctnmth) %>%
# Keep only rows with differences > 0 in variables
filter(if_all(any_of(vars_RF_slope_Landsat[grepl("diff",
                                                    vars_RF_slope_Landsat)]),
              ~ .x > 0)) %>%
# Rough validation based on NDWI
rough_validation_Landsat() %>%
nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 117329
```

```

# Landsat months
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
# Data filtering
# Remove rows with missing values in predictors
dplyr::filter(if_all(all_of(vars_RF_months_Landsat), ~ !is.na(.))) %>%
# Remove all rows with wrong values of indices (not between -1 and 1)
dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
# With Landsat data, many observations removed because of high/low EVI values
# In Landsat_extraction, values of EVI outside of expected range were set as NA,
# so better not to use EVI here
# dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
# Join ReSurvey data to get Location method
left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
# Select only variables needed
dplyr::select(PlotObservationID, EUNIS1, all_of(vars_RF_months_Landsat),
              Lctnmth) %>%
# Keep only rows with differences > 0 in variables
filter(if_all(any_of(vars_RF_months_Landsat[grepl("diff",
                                                    vars_RF_months_Landsat)]),
              ~ .x > 0)) %>%
# Rough validation based on NDWI
rough_validation_Landsat() %>%
nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 100990
```

```

# Landsat 2
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
# Data filtering
# Remove rows with missing values in predictors
dplyr::filter(if_all(all_of(vars_RF_2_Landsat), ~ !is.na(.))) %>%
# Remove all rows with wrong values of indices (not between -1 and 1)

```

```

dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
# With Landsat data, many observations removed because of high/low EVI values
# In Landsat_extraction, values of EVI outside of expected range were set as NA,
# so better not to use EVI here
# dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
# Join ReSurvey data to get Location method
left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
# Select only variables needed
dplyr::select(PlotObservationID, EUNIS1, all_of(vars_RF_2_Landsat),
              Lctnmth) %>%
# Keep only rows with differences > 0 in variables
filter(if_all(any_of(vars_RF_2_Landsat[grepl("diff",
                                              vars_RF_2_Landsat)]),
              ~ .x > 0)) %>%
# Rough validation based on NDWI
rough_validation_Landsat() %>%
nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 117331
```

N obs GPS-lcoated

```

# Landsat slope
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
# Join ReSurvey data to get Location method
left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
# Rough validation based on NDWI
rough_validation_Landsat() %>%
filter_data_Landsat(vars_RF_slope_Landsat) %>%
nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 92776
```

```

# Landsat months
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
# Join ReSurvey data to get Location method
left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
# Rough validation based on NDWI
rough_validation_Landsat() %>%
filter_data_Landsat(vars_RF_months_Landsat) %>%
nrow()

```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 82220
```

```
# Landsat 2
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  filter_data_Landsat(vars_RF_2_Landsat) %>%
  nrow()
```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 92777
```

N obs after NDVI-filtering

```
# Landsat slope
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  filter_data_Landsat(vars_RF_slope_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value") %>%
  nrow()
```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 83497
```

```
# Landsat months
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  filter_data_Landsat(vars_RF_months_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value") %>%
  nrow()
```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 73997
```

```
# Landsat 2
data_validation_Landsat %>%
  filter(if_any(c(2:80, 84:116), ~ !is.na(.))) %>%
  # Join ReSurvey data to get Location method
  left_join(ReSurvey_data %>% select(PlotObservationID, Lctnmth)) %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  filter_data_Landsat(vars_RF_2_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value") %>%
  nrow()
```

```
## Joining with 'by = join_by(PlotObservationID)'
```

```
## [1] 83498
```

Session info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Spanish_Spain.utf8  LC_CTYPE=Spanish_Spain.utf8
## [3] LC_MONETARY=Spanish_Spain.utf8 LC_NUMERIC=C
## [5] LC_TIME=Spanish_Spain.utf8
##
## time zone: Europe/Madrid
## tzcode source: internal
##
## attached base packages:
## [1] stats4      grid        parallel    stats       graphics    grDevices    utils
## [8] datasets    methods     base
##
## other attached packages:
## [1] scales_1.4.0      patchwork_1.3.2    gghalves_0.1.4
## [4] permimp_1.1-0     beeper_2.0         rlang_1.1.7
## [7] pROC_1.19.0.1     randomForest_4.7-1.2 caret_7.0-1
## [10] lattice_0.22-7    moreparty_0.4.2    party_1.3-18
## [13] strucchange_1.5-4 sandwich_3.1-1     zoo_1.8-15
## [16] modeltools_0.2-24 mvtnorm_1.3-3      doParallel_1.0.17
## [19] iterators_1.0.14  foreach_1.5.2      here_1.0.2
## [22] lubridate_1.9.5   forcats_1.0.1      stringr_1.6.0
```

```

## [25] dplyr_1.2.0          purrr_1.2.1          readr_2.1.6
## [28] tidyr_1.3.2          tibble_3.3.1         ggplot2_4.0.2
## [31] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] magrittr_2.0.4        multcomp_1.4-29      otel_0.2.0
## [4] e1071_1.7-17          matrixStats_1.5.0    compiler_4.5.2
## [7] systemfonts_1.3.1     reshape2_1.4.5       vctrs_0.7.1
## [10] crayon_1.5.3          pkgconfig_2.0.3      fastmap_1.2.0
## [13] labeling_0.4.3        utf8_1.2.6           promises_1.5.0
## [16] phosphoricons_0.2.1   rmarkdown_2.30       prodlim_2025.04.28
## [19] tzdb_0.5.0            ragg_1.5.0           bit_4.6.0
## [22] xfun_0.56             cachem_1.1.0         jsonlite_2.0.0
## [25] recipes_1.3.1         later_1.4.5          R6_2.6.1
## [28] coin_1.4-3            bslib_0.10.0         stringi_1.8.7
## [31] RColorBrewer_1.1-3    parallelly_1.46.1    rpart_4.1.24
## [34] jquerylib_0.1.4       Rcpp_1.1.1           knitr_1.51
## [37] future.apply_1.20.1   rclipboard_0.2.1     audio_0.1-12
## [40] httpuv_1.6.16         Matrix_1.7-4         splines_4.5.2
## [43] nnet_7.3-20           varImp_0.4           timechange_0.4.0
## [46] tidyselect_1.2.1      rstudioapi_0.18.0    yaml_2.3.12
## [49] timeDate_4052.112     codetools_0.2-20     listenv_0.10.0
## [52] plyr_1.8.9            shiny_1.12.1         withr_3.0.2
## [55] S7_0.2.1              evaluate_1.0.5       future_1.69.0
## [58] survival_3.8-6        proxy_0.4-29         pillar_1.11.1
## [61] DT_0.34.0             generics_0.1.4       vroom_1.7.0
## [64] rprojroot_2.1.1       hms_1.1.4            globals_0.19.0
## [67] xtable_1.8-4          class_7.3-23         glue_1.8.0
## [70] tools_4.5.2           data.table_1.18.2.1  ModelMetrics_1.2.2.2
## [73] gower_1.0.2           measures_0.3          libcoin_1.0-10
## [76] ipred_0.9-15          nlme_3.1-168         cli_3.6.5
## [79] textshaping_1.0.4     lava_1.8.2           gtable_0.3.6
## [82] sass_0.4.10           digest_0.6.39        TH.data_1.1-5
## [85] htmlwidgets_1.6.4     farver_2.1.2         htmltools_0.5.9
## [88] lifecycle_1.0.5       hardhat_1.4.2        shinyWidgets_0.9.0
## [91] mime_0.13             bit64_4.6.0-1        MASS_7.3-65

```